EXERCISE 4 : SQL JOINS

1. INNER JOIN

Table: students

| student_id | student_name |
|---|---|
| 1 | Alice |
| 2 | Bob |
| 3 | Charlie |

Table: grades

| student_id | grade |
|---|---|
| 2 | B |
| 3 | A |
| 4 | C |

Join students and grades to display only students who have grades.

- student_id student_name grade.

```
SELECT A.student_id, student_name, grade
FROM student AS A
INNER JOIN grades AS B
ON A.student_id = B.student_id;
```

| student_id | student_name | grade |
|---|---|---|
| 2 | Bob | B |
| 3 | Charlie | A |

2. LEFT JOIN

Display all employees and departments they belong to. Include employees with no department

Table: employees

| emp_id | emp_name |
|---|---|
| 1 | John |
| 2 | Lisa |
| 3 | Mike |

Table: departments

| emp_id | dept_name |
|---|---|
| 2 | HR |
| 4 | IT |

```
SELECT A.emp_id, emp_name, dept_name
FROM employees AS A
LEFT JOIN department AS B
ON A.emp_id = B.emp_id;
```

| emp_id | emp_name | department_name |
|---|---|---|
| 1 | John | Null |
| 2 | Lisa | HR |
| 3 | Mike | Null |

## 3. FULL OUTER JOIN

Display a complete list of products and their quantities sold. Include products with no sales and sales for unknown products.

Table : products

| product_id | product_name |
|---|---|
| 1 | Laptop |
| 2 | Mouse |
| 3 | Keyboard |

Table : sales

| product_id | quantity |
|---|---|
| 2 | 50 |
| 4 | 30 |

```
SELECT COALESCE(A.product_id, B.product_id) AS prod_id
product_name, quantity
FROM products AS A
FULL OUTER JOIN sales AS B
ON A.product_id = B.product_id;
```

| product_id | product_name | quantity |
|---|---|---|
| 1 | laptop | Null |
| 2 | Mouse | 50 |
| 3 | Keyboard | Null |
| 4 | Null | 30 |

## 4. LEFT JOIN + CASE

Display all orders and indicate whether the customer is 'New' or 'Returning'

→ order_id customer_id amount customer_name customer_type (New customer / Returning customer)

Table: orders

| order_id | customer_id | amount |
|---|---|---|
| 1 | 101 | 500 |
| 2 | 102 | 300 |
| 3 | 105 | 0 |

Table: customers

| customer_id | customer_name |
|---|---|
| 101 | Paul |
| 102 | Sarah |
| 104 | Emma |

```
SELECT order_id, A.customer_id, amount, customer_name
CASE
WHEN B.customer_id IS NOT NULL THEN 'Returning customer'
ELSE 'New customer'
END AS customer_type
FROM orders AS A
LEFT JOIN customers AS B
ON A.customer_id = B.customer_id
```

| order_id | customer_id | amount | customer_name | type |
|---|---|---|---|---|
| 1 | 101 | 500 | Paul | Returning |
| 2 | 102 | 300 | Saidh | Returning |
| 3 | 105 | 0 | Null | New |

## 5. LEFT JOIN + GROUP BY + SUM

Show total sales per region and include regions with no sales

⇒ region_id region_name total_sales

Table: sales

| sale_id | region_id | amount |
|---|---|---|
| 1 | 1 | 2000 |
| 2 | 2 | 3500 |
| 3 | 4 | 1000 |

Table: regions

| region_id | region_name |
|---|---|
| 1 | North |
| 2 | South |
| 3 | East |

```
SELECT A.region_id, region_name, sum(amount) AS total_sales
from region AS A
LEFT JOIN sales AS B
ON A.region_id = B.region_id
GROUP BY A.region_id, region_name;
```

| region_id | region_name | total sales |
|---|---|---|
| 1 | North | 2000 |
| 2 | South | 3500 |
| 3 | East | Null |

## 6. LEFT JOIN + CASE

Classify students based on attendance

⇒ student_id name days_present attendance_status (Excellent / Needs Improvement / Poor attendance)

Table: students

| student_id | name |
|---|---|
| 1 | Alice |
| 2 | Bob |
| 3 | Charlie |

table: attendance

| student_id | days_present |
|---|---|
| 1 | 18 |
| 2 | 5 |
| 4 | 20 |

```
SELECT A.student_id, name, days_present
CASE
WHEN days_present >= 15 THEN 'EXCELLENT'
WHEN days_present BETWEEN 6 AND 14 THEN 'NEEDS IMPR
WHEN days_present <= 5 THEN 'Poor attendance'
ELSE 'No record'
END AS attendance_status
FROM students AS A
LEFT JOIN attendance AS B
ON A.student_id = B.student_id
```

| student_id | name | days_present | attendance_status |
|---|---|---|---|
| 1 | Alice | 18 | Excellent |
| 2 | Bob | 5 | Poor attendance |
| 3 | Charlie | Null | No record |

## 7. INNER JOIN + COUNT + GROUP BY

Show number of tasks per project. Only include projects that have tasks.

→ project_id name task_count

projects

| project_id | name |
|---|---|
| 1 | AI ChatBot |
| 2 | Website |
| 3 | Data report |

tasks

| task_id | project_id | status |
|---|---|---|
| 10 | 1 | Complete |
| 11 | 1 | Pending |
| 12 | 2 | complete |

```
SELECT A.project_id, name, COUNT(task_id) AS task_count
FROM projects AS A
INNER JOIN task AS B
ON A.project_id = B.project_id
GROUP BY A.project_id, name;
```

| project_id | name | task_count |
|---|---|---|
| 1 | AI Chatbot | 2 |
| 2 | Website | 1 |

## 8. FULL OUTER JOIN + CASE + WHERE

Classify customers based on whether they returned anything and filter by high order total

⇒ cust_id order_total return_total return_status (Return No return)

Table: orders

| order_id | cust_id | order_total |
|---|---|---|
| 1 | 11 | 120 |
| 2 | 12 | 250 |
| 3 | 13 | 180 |

Table: Returns

| return_id | cust_id | return_total |
|---|---|---|
| 101 | 11 | 20 |
| 102 | 14 | 100 |

```
SELECT COALESCE(A.cust_id, B.cust_id) AS cust_id
    order_total, return_total
CASE
WHEN return_total IS NOT NULL THEN 'Returned'
ELSE 'No Return'
END AS Return_status
FROM orders AS A
FULL OUTER JOIN returns AS B
ON A.cust_id = B.cust_id
```

WHERE order_total > 100 )

| user_id | order_total | return_total | return_status |
|---|---|---|---|
| 11 | 120 | 20 | Return |
| 12 | 250 | NULL | No return |
| 13 | 180 | NULL | No return |

## LEFT JOIN + COUNT + ORDER BY

Count how many times each user logged in.

Table: Users

| user_id | name |
|---|---|
| 1 | Nelson |
| 2 | Gloria |
| 3 | Steve |

Table: Logins

| user_id | login_date |
|---|---|
| 2 | 2024-06-01 |
| 2 | 2024-06-02 |
| 3 | 2024-06-03 |

- user_id name login_count Order by login_count DESC

```
SELECT A.user_id, name, count(login_date) AS
login_count
FROM users AS A
LEFT JOIN logins AS B
ON A.user_id = B.user_id
Group BY A.user_id, name
ORDER BY login_count DESC;
```

| user_id | name | login_count |
|---|---|---|
| 2 | Gloria | 2 |
| 3 | Steve | 1 |
| 1 | Nelson | 0 |

## 10. LEFT JOIN + CASE + ORDER BY

Show all teachers and the subject they teach. If no subject label appropriately.

→ teacher_id teacher_name subject_name ( or 'No subj assig)

ORDER BY teacher_name DESC

Table : teachers

| teacher_id | teacher_name |
|---|---|
| 1 | Mr Mlongwane |
| 2 | Ms Ndaba |
| 3 | Mr Dlamini |

Table : Subjects.

| subject_id | teacher_id | subject_name |
|---|---|---|
| 1 | 1 | Math |
| 2 | 1 | Science |
| 3 | 4 | History |

SELECT A.teacher_id, teacher_name, COALESCE (subject_name, 'No subject assigned) AS subject_name

FROM teachers AS A

LEFT JOIN subjects AS B

ON A.teacher_id = ON.B ~~teacher~~_id

ORDER BY teacher_name ASC ;

| teacher_id | teacher_name | subject_name |
|---|---|---|
| 3 | Mr Dlamini | No subject Assigned |
| 1 | Mr Mlongwane | Math |
| 1 | Mr Mlongwane | Science |
| 2 | Ms Ndaba | No subject assigned |